



RESEARCH ARTICLE

10.1029/2024MS004766

Key Points:

- A simplified Additive Gaussian Processes method is designed for emulating sparselysampled climate model perturbed parameter ensembles (PPEs)
- The method, tested on NCAR CAM and GISS ModelE PPEs, exhibits performance similar to other emulators, but with less dependence on hyperparameters
- The method provides new diagnostics on parameter–output importance not captured in linear correlation maps, enabling new insights that could benefit the future design of PPEs and emulators

Correspondence to:

Q. Yang,
qy2288@columbia.edu

Citation:

Yang, Q., Elsaesser, G. S., van Lier-Walqui, M., & Eidhammer, T. (2025). A simple emulator that enables interpretation of parameter–output relationships, applied to two climate model PPEs. *Journal of Advances in Modeling Earth Systems*, 17, e2024MS004766. <https://doi.org/10.1029/2024MS004766>

Received 8 OCT 2024
Accepted 13 APR 2025

Author Contributions:

Conceptualization: Qingyuan Yang, Gregory S. Elsaesser, Marcus van Lier-Walqui, Trude Eidhammer
Data curation: Qingyuan Yang, Gregory S. Elsaesser, Marcus van Lier-Walqui, Trude Eidhammer
Formal analysis: Qingyuan Yang
Funding acquisition: Gregory S. Elsaesser, Marcus van Lier-Walqui
Investigation: Gregory S. Elsaesser, Marcus van Lier-Walqui, Trude Eidhammer
Methodology: Qingyuan Yang

© 2025 The Author(s). Journal of Advances in Modeling Earth Systems published by Wiley Periodicals LLC on behalf of American Geophysical Union. This is an open access article under the terms of the [Creative Commons Attribution-NonCommercial License](#), which permits use, distribution and reproduction in any medium, provided the original work is properly cited and is not used for commercial purposes.

A Simple Emulator That Enables Interpretation of Parameter-Output Relationships, Applied to Two Climate Model PPEs

Qingyuan Yang^{1,2} , Gregory S. Elsaesser^{1,3,4} , Marcus van Lier-Walqui^{1,3,5} , and Trude Eidhammer⁶

¹Learning the Earth with Artificial Intelligence and Physics (LEAP) National Science Foundation (NSF) Science and Technology Center, Columbia University, New York, NY, USA, ²Department of Earth and Environmental Engineering, Columbia University, New York, NY, USA, ³NASA Goddard Institute for Space Studies, New York, NY, USA, ⁴Department of Applied Physics and Applied Mathematics, Columbia University, New York, NY, USA, ⁵Center for Climate System Research, Columbia University, New York, NY, USA, ⁶NSF National Center for Atmospheric Research, Boulder, CO, USA

Abstract We present a new additive method, referred to as *sage* for Simplified Additive Gaussian processes Emulator, for emulating climate model Perturbed Parameter Ensembles (PPEs). *sage* estimates the value of a climate model output as the sum of additive terms. Each additive term is the mean of a Gaussian Process, and corresponds to the impact of a parameter or parameter group on the variable of interest. This design caters to the sparsity of PPEs, which are characterized by limited ensemble members and high dimensionality of the parameter space and raise the issue of parameter sensitivity in the emulator design. *sage* quantifies the variability explained by different parameters and parameter groups, providing additional insights on the parameter–climate model output relationship. We apply *sage* to two climate model PPEs and compare it to a fully connected Neural Network. The two methods have comparable performance with both PPEs, but *sage* provides insights on parameter and parameter group importance as well as diagnostics useful for optimizing PPE design. Insights gained from applying the method and comparing its performance with Neural Network are pointed out which have not been previously addressed. Our work highlights that analyzing the PPE used to train an emulator is different from analyzing data generated from an emulator trained on the PPE, as the former provides more insights on the data structure in the PPE which could help inform the emulator design. Our work also proposes new questions on the optimal way of working with climate model PPEs.

Plain Language Summary Climate models have many parameters whose values are uncertain. A parameter could be one that affects the fall speed of snow in the climate models. Researchers design Perturbed Parameter Ensembles (PPEs) to study how the parameters affect the model output (e.g., precipitation) so as to gain insight on the best settings for parameters. A PPE is a set of climate model runs (i.e., the ensemble members; normally a few hundred) with different parameter values. Since running climate models is very computationally expensive, all parameter combinations cannot be run. In this work, we propose a “surrogate model” or “emulator” method that, once properly trained, estimates the climate model output climatologies given a set of model parameters that have not been run by the climate model. The method is additive, meaning that the impacts of parameters and parameter interactions on the climatologies are assumed additive. The method analyzes the relationship between the parameters and climatologies in a PPE and makes predictions at the same time. We apply our method to two climate model PPEs, and compare its performance with that of fully connected Neural Networks. The performance of the two methods is comparable, and additional insights can be obtained.

1. Introduction

Earth System Model (ESM) perturbed parameter ensembles (PPEs) are useful tools for analyses of ESM emergent property or performance score sensitivities to physics parameter choices (e.g., Murphy et al. (2004), Collins et al. (2011), Williamson et al. (2013), Parker (2013), Haughton et al. (2014), D. Sexton et al. (2019), Duffy et al. (2024), Eidhammer et al. (2024), Elsaesser et al. (2025), Gettelman et al. (2024)) as well for broadly quantifying envelopes of projection uncertainty (Hourdin et al., 2023). PPEs are generated by running a climate model with different sets of parameter values. Latin Hypercube Sampling (LHS; McKay et al. (2000)), which is

Project administration: Gregory S. Elsaesser, Marcus van Lier-Walqui
Software: Qingyuan Yang, Marcus van Lier-Walqui, Trude Eidhammer
Validation: Qingyuan Yang, Gregory S. Elsaesser, Marcus van Lier-Walqui, Trude Eidhammer
Visualization: Qingyuan Yang, Marcus van Lier-Walqui
Writing – original draft: Qingyuan Yang
Writing – review & editing: Qingyuan Yang, Gregory S. Elsaesser, Marcus van Lier-Walqui, Trude Eidhammer

similar to random sampling but ensures that the sampled parameter marginal distributions are uniform, is commonly used for generating the parameter values (e.g., Lambert et al. (2013), Li et al. (2019), Eidhammer et al. (2024)), although there are other more sophisticated sampling approaches (Cleary et al., 2021; Dunbar et al., 2021; Elsaesser et al., 2025; Lopez-Gomez et al., 2022).

PPEs commonly serve as the training data for building climate model emulators (e.g., Hawkins et al. (2019), D. Sexton et al. (2019), Dagon et al. (2020), Tran et al. (2021), Gettelman et al. (2022), Beucler et al. (2022), Elsaesser et al. (2025)). Once properly trained, an emulator can approximate the outputs of a computationally expensive climate model at a negligible cost. The emulator can then propagate climate model outputs for various purposes such as parameter estimation (Carzon et al., 2023; Dagon et al., 2020; Elsaesser et al., 2025; Fletcher et al., 2022; Hourdin et al., 2017; Li et al., 2019; Peatier et al., 2022; van Lier-Walqui et al., 2020; Watson-Parris et al., 2021), sensitivity analysis (Proske et al., 2022; D. Sexton et al., 2019; Webb et al., 2013), and uncertainty propagation (Hourdin et al., 2023; Hutchins et al., 2016).

Different methods have been developed for analyzing and emulating climate model PPEs. These include linear regression (e.g., Duffy et al. (2024)), Gaussian Processes (GPs; Lee et al. (2011), L. Regayre et al. (2014), J. Johnson et al. (2015), Yang et al. (2020), Carzon et al. (2023)) and Neural Network (NN; Reichstein et al. (2019), Labe and Barnes (2021), Dagon et al. (2022)) frameworks. Different emulation methods have also been integrated into a single framework, providing the flexibility of choosing the optimal method after inter-comparison (e.g., Watson-Parris et al. (2021)) as well as providing information on emulator structural uncertainty.

With the increasing use of PPEs, we need not only reliable emulators but also tools that analyze some of the nuanced (and non-linear) structure existing in climate model PPEs. This is an important consideration as more sophisticated analysis tools not only aid in PPE interpretation and emulation, they also can inform on future design and sampling considerations for subsequent PPEs, particularly given the known limitations with sampling in high dimensional state spaces. Additionally, traditional approaches for examining the parameter-target variable (i.e., the model output or model skill score) relationships have limitations. For example, the popular “heat map” revealing the pairwise correlation between parameters and target variables does not consider the effect of parameter interaction. (This issue is recognized, and studies (e.g., Kennedy et al. (2024)) have used different measures to quantify parameter importance.) In choosing and designing an emulator, we cannot easily and explicitly explain how emulator performance is affected by different qualities (e.g., the number of ensemble members) of the analyzed PPE. For example, we do not know whether emulating the target variables all at once or a group at a time impacts emulator performance without brute force manual testing.

Some of the aforementioned issues can be addressed by analyzing a large number of samples generated by a trained emulator. However, all interpretations from the analysis are conditioned on the trained emulator. The introduced and additional error from the imperfect emulation is assumed aleatoric (e.g., as pointed out in Kennedy et al. (2024)). This neglects the possibility that the emulator performance can be further improved (e.g., as proved later in this work, by grouping the target variables and emulate them separately) by analyzing the PPE data set (as opposed to analysis on the generated samples).

Toward addressing the unknowns listed above, in this work we propose a simple method that (a) serves as a fast and interpretable emulator and (b) a data analysis tool for climate model PPEs. The method captures the low-frequency structure between parameters and target variables (analogous to a low-pass filter) and assumes that such low-frequency structure can be captured as the sum of additive terms. The expectation of this method is not to outperform existing emulator methods but to achieve the above two points using a simple method at a low cost in terms of emulation fidelity compared to other emulator methods such as NN.

We compare the method performance against a NN that was recently used in studies pertaining to two diverse PPEs (Eidhammer et al., 2024; Elsaesser et al., 2025). Our new method aims at capturing the non-local and low-frequency relationship between parameters and target variables by assuming it to be the adequate ansatz. It is designed to cater to the sparsity (i.e., small ensemble size relative to the number of parameters) of typical climate model PPEs.

The method is simplified from additive Gaussian Processes (Cheng et al., 2019; Duvenaud et al., 2011; Plate, 1999) but with modifications that enable improved performance for sparsely sampled state spaces. We term the method *sage* for Simplified Additive Gaussian processes Emulator. We refer to the method *sage* or “the (present or current) method” interchangeably in the following text. The method resembles the polynomial-based

emulator described in Williamson et al. (2013) in its way of selecting the effective parameters. It sequentially and iteratively identifies the parameters and parameter groups that a target variable is sensitive to, and models their impact on the target variable. The sum of these impacts (i.e., additive terms) represents the emulator prediction.

sage estimates the target variable as the sum of multiple terms, each corresponding to a parameter or parameter groups. It quantifies and outputs how much the Root-Mean-Square Error (RMSE) is gradually reduced after each term is added to the emulator prediction. The reduced RMSE quantifies how individual parameters and parameter groups contribute to the emulator performance, providing insights on their impact on the target variable.

We apply *sage* to two PPEs generated from different climate models: (a) the latest Goddard Institute for Space Studies (GISS) ESM ModelE3 (Cesana et al., 2019; Elsaesser et al., 2025; Russotto et al., 2022) and (b) the Community Earth System Model version 2 (CESM2; Danabasoglu et al. (2020)) Community Atmosphere Model 6 (CAM6; Gettelman et al. (2019)). We compare emulator performance against a NN emulator used in studies of the same two PPEs (i.e., from Elsaesser et al. (2025) and Eidhammer et al. (2024)), before extension to the PPE-structural-analysis benefit provided by *sage*. Our work reveals previously-unnoticed insights that would benefit future PPE analysis and emulator design. The comparative study using both *sage* and NN suggests that the insights are likely not method-specific. Highlights of such insights include: (a) neither emulating the variables all at once nor one-at-a-time is (always) the perfect emulator design; (b) a group of less-sensitive parameters could have a non-negligible and cumulative impact on the overall emulator performance. We thus need to be *very* careful in determining whether a parameter is important or not; and (c) the emulator performance does not change linearly with the number of ensemble members for training.

The analysis, comparison, and findings based on the application of two emulator methods (*sage* and a NN) to two different PPEs are considered equally or even more important than the presented method itself, as they inform the design of PPEs and climate model emulators for future studies.

2. Data Sets

We leverage two very different PPEs in this work to demonstrate the value of *sage*. The two PPEs are fully described in Elsaesser et al. (2025) and Eidhammer et al. (2024), respectively. They are briefly introduced here.

The ModelE3 PPE has 851 members, with each member characterized by a unique set of 45 physics parameters spanning the atmospheric convection, large-scale cloud, and turbulence modules of ModelE3. Their ranges, summarized in Table 1 of Elsaesser et al. (2025), were determined based on modeler expertise as in D. M. Sexton et al. (2021). Each parameter combination is used in a short 1-year, atmosphere-only simulation (110 vertical layers, $2.5 \times 2.0^\circ$ horizontal resolution) run with a present-day prescribed sea surface temperature (SST) climatology and atmospheric composition. The first 451 (450 plus one default run) PPE members were generated using parameter combinations provided by LHS. The parameters of the next 100 PPE members were generated using random sampling. The final 300 sets of parameters were sampled *sequentially* (100 at a time for three times) based on a method detailed in Elsaesser et al. (2025) that aims to generate a special type of PPE (termed a “calibrated physics ensemble,” or CPE) whose members agree with numerous earth energy and water cycle diagnostics. This 300 member CPE relied on machine learning and a Markov Chain Monte Carlo method to sample from a target probability distribution defined by comparison of the emulated model with information from numerous observational products. As a result, relative to the first 551 members, the three 100-member batches yield outputs exhibiting progressively reduced biases with respect to satellite observations. In this work, we exclude the last 100 ensemble members in the event that they are too clustered near an hypervolume in the parameter space, which might affect the emulator performance (i.e., the output variables of these 100 ensemble members tend to be very similar with each other, hence whether and how many of them are included in the training and validation data sets would greatly affect the method performance).

CESM2 (Danabasoglu et al., 2020) CAM6 (Gettelman et al., 2019) serves as the base model for generation of the second PPE used in this work. Hereafter, we often refer to this PPE as the CAM6 PPE. The CAM6 PPE has 262 members with 45 parameters perturbed and sampled using LHS. These parameters span the unified turbulence closure (CLUBB; Golaz et al. (2002)), cloud microphysics (MG2; Gettelman and Morrison (2015)), Modal Aerosol Model (MAM; Liu et al. (2012)), and the Zhang-McFarlane deep convection (ZM; Zhang and McFarlane (1995)) schemes. Their ranges are determined with expert elicitation, and can be found in Table 1 of Eidhammer et al. (2024) (also not listed here for simplicity). Two pairs (*clubb_C6rt* and *clubb_C6thl*; *clubb_C6rtb*

Table 1
Variables of Interest in This Work

Variable name	Processed model output or model skill (all scalar ^a)	Description	Whether calculated with ModelE3/CAM6 PPE
StreamFunction_DJF	–	Hadley Cell stream function scalar magnitudes for DJF	Yes/No
StreamFunction_JJA	–	Hadley Cell stream function scalar magnitudes for JJA	Yes/No
netrad_toa	Global average	Net radiation at TOA	Yes/Yes ^d
netheat_grnd	Global average	Net heat into the ground	Yes/No
albedo	Global average	SW albedo	Yes/Yes
swabstoa_ave	Global average	TOA absorbed SW radiation	Yes/Yes
olr_ave	Global average	TOA outgoing LW radiation	Yes/Yes
T_Trop	Regional average	Temperature at the tropopause	Yes/No
swabstoa	Metric based on global zonal climatologies	TOA SW absorbed radiation	Yes/Yes ^d
sw_cre	Metric based on global zonal climatologies	TOA SW cloud radiative forcing	Yes/Yes ^d
olr	Metric based on global zonal climatologies	Outgoing LW radiation	Yes/Yes ^d
lw_cre	Metric based on global zonal climatologies	TOA LW cloud radiative forcing	Yes/Yes ^d
prec_mc	Metric based on global zonal climatologies	Surface convective precipitation rate	Yes/Yes
tcc	Metric based on global zonal climatologies	Total cloud cover	Yes/Yes
tcc_isccp	Metric based on global zonal climatologies	Total cloud cover from the ISCCP simulator	Yes/Yes
tcc_shcu	Metric based on global zonal climatologies	Shallow cumulus cloud cover	Yes/No
tcc_sc	Metric based on global zonal climatologies	Stratocumulus cloud cover	Yes/No
T_for_Phase0.90	Metric based on global zonal climatologies	Average temperature as a function of latitude from 80°S to 80°N, for which 90% of the condensate is classified as ice	Yes/No
Z_for_Phase0.90	Metric based on global zonal climatologies	Average altitude as a function of latitude from 80°S to 80°N, for which 90% of the condensate is classified as ice	Yes/No
swabstoa_soceans	Regional bias	Longitudinally-averaged absorbed SW radiation for latitude bands from 45°S to 75°S (southern ocean)	Yes/Yes
pwv	Metric based on global zonal climatologies ^b	Column water vapor	Yes/Yes ^d
qv_200hpa	Metric based on global zonal climatologies ^b	Specific humidity at 200 hpa	Yes/Yes ^d
qv_600hpa	Metric based on global zonal climatologies ^b	Specific humidity at 600 hpa	Yes/Yes ^d
qv_925hpa	Metric based on global zonal climatologies ^b	Specific humidity at 925 hpa	Yes/Yes
T_200hpa	Metric based on global zonal climatologies ^b	Temperature humidity at 200 hpa	Yes/Yes ^d
T_600hpa	Metric based on global zonal climatologies ^b	Temperature humidity at 600 hpa	Yes/Yes ^d
T_925hpa	Metric based on global zonal climatologies ^b	Temperature humidity at 925 hpa	Yes/Yes
TLWP (GISS ModelE3) and Filtered CLWP (CAM6)	Metric based on global zonal climatologies ^b	Total (cloud + rain) liquid water path and filtered cloud liquid water path ^c	Yes/Yes ^d
TIWP	Metric based on global zonal climatologies ^b	Total ice water path	Yes/No
prec	Metric based on global zonal climatologies ^b	Surface precipitation rate	Yes/Yes ^d
cdnc	Metric based on global zonal climatologies ^b	Cloud top droplet number concentration	Yes/No

Table 1
Continued

Variable name	Processed model output or model skill (all scalar ^a)	Description	Whether calculated with ModelE3/CAM6 PPE
sfcwind	Metric based on global zonal climatologies ^b	Surface wind	Yes/No
prec_amazon	Regional bias	Amazon-basin precipitation	Yes/No

Note. SW: shortwave; LW: longwave; TOA: Top of the atmosphere. ^aCalculated as weighted (determined by latitude) sum of zonal climatologies or scores. ^bThese variables have more than one observational products. See Elsaesser et al. (2025) for more details on how they are calculated, which does not affect any arguments in this work. ^cThe filtering is to make the satellite products comparable to the CAM6 output. ^dGlobal average used for emulation in Figure 4c.

and *clubb_C6htlb*) of CLUBB parameters are perturbed and sampled simultaneously, reducing the number of free parameters to 43. Each PPE member simulation is run using near present day cyclic boundary conditions for year 2000. The greenhouse gases and atmospheric oxidants take the average values of the 1995–2005 period. The average monthly SSTs during 1995–2010 are used. The emission of aerosols and precursors is set to 1995–2005 in these present day simulations. The simulation resolution is 0.9° (latitude) × 1.1° (longitude) with 32 vertical levels (maximum at 10 hPa). Each simulation runs for a period of 3 years.

2.1. Target Variable Calculation

Following Elsaesser et al. (2025), we choose 33 target variables to emulate for the ModelE3 PPE. They are all averaged over full 1-year simulation. Eight of them are calculated based on global or regional model outputs (e.g., top of the atmosphere outgoing long wave radiation or Amazon rainfall), and the rest denote model performance scores, which are functions of model outputs and satellite observations. Following Equation 1 of Elsaesser et al. (2025), one such performance score (*C*) is calculated as:

$$C = \sum \left(\frac{|M_{ilat} - O_{ilat}^H| + |M_{ilat} - O_{ilat}^L| + (O_{ilat}^H - O_{ilat}^L)}{2} \times W_{ilat} \right)^2, \quad (1)$$

where M_{ilat} is the model output at the (*ilat*)th latitude band. O_{ilat}^H and O_{ilat}^L denote the largest and smallest satellite observations at the corresponding latitude band, respectively. If there are no multiple satellite observations, $O_{ilat}^H = O_{ilat}^L$. W_{ilat} denotes a weighting factor based on the cosine of latitude. The novelty in Equation 1 is that the systematic error (i.e., the observations of different satellite products at the same location or latitude band being inconsistent) from different satellite products is taken into account in the metric calculation. All outputs are described in greater detail in Elsaesser et al. (2025).

Three variables (i.e., *T_100hpa*, *qv_100hpa* and *Trop_Cyclone_Count*) used in Elsaesser et al. (2025) are excluded here because the former two have very few non-zero values given the uncertainty in the observations for these variables exceeding the typical PPE-member simulated output (see Figure 4e of Elsaesser et al. (2025)), while the last output variable is not continuous. Since some output variables from the ModelE3 PPE are not available in the CAM6 PPE, we have 21 target variables for the latter averaged over three-year simulation. This does not affect any interpretations or arguments derived from our analyses. A brief description of the 33 target variables and whether they are calculated based on the CAM6 PPE are given in Table 1. It is noted that there is redundancy in the selection of these target variables. See Elsaesser et al. (2025) for the corresponding justifications. This is not within the context of the present work.

3. Method

Climate model PPEs are commonly characterized by high dimensional parameter spaces (e.g., several tens of parameters are common) and limited numbers of ensemble members (e.g., in most cases less than 1000), implying sparsely-sampled parameter spaces. The sparsity makes the task of emulating PPE data sets analogous to drawing

the coastline of a country given three points on a map. Due to the sparsity, we assume that high-frequency relationships between the parameters and the target variables exist, but cannot be robustly captured.

We fix the hyper-parameters instead of learning them, because they cannot be computed robustly from the typical small ensemble size of PPE data sets. For example, we have used more conventional GP method with different libraries to emulate the data sets used in this work, and one of them often produces overfitted results (as indicated by very small *range* hyperparameter, see text below for explanation on this hyperparameter), and another one gives relatively more reliable results only when the initial guesses on some hyperparameters are within certain ranges.

sage is a simplified technique that utilizes aspects of additive GPs (Cheng et al., 2019; Duvenaud et al., 2011; Lu et al., 2022). It is made available on Zenodo (<https://doi.org/10.5281/zenodo.14853174>; Yang et al. (2025)). For a commonly used GP emulator that is non-additive, only one GP is trained and used for the prediction. Such a single GP is a function of all parameters. Conversely, for an additive GP emulator, its prediction is the sum of multiple GPs, with each GP being a function of different parameters and (or) parameter groups. In *sage*, following the considerations and arguments mentioned above, we do not utilize a formal additive GP for simplicity (i.e., we do not estimate the GP hyperparameters). We note that the values of the GP hyperparameters would not greatly affect the *sage* performance especially when the target variables are global climatologies. This marks a key finding from this work, which we further assess later.

3.1. Additive Emulator

sage treats one climate model output at a time. We denote one scalar output of a climate model $f(\cdot)$ as z and the D -dimensional parameter space as $z = f(x_1, x_2, \dots, x_D)$ with $x_1, x_2, \dots, x_D$ being the model parameters. For one target variable, our emulator decomposes z into additive terms, with each a function of one, two or three parameters, as follows:

$$f(x_1, x_2, \dots, x_D) \approx \sum_{j=1}^{M_1} h_{1,j}(x_{I_{1,j}}) + \sum_{k=1}^{M_2} h_{2,k}(X_{2,k}) + \sum_{l=1}^{M_3} h_{3,l}(X_{3,l}), \quad (2)$$

where $h_{1,j}(x_{I_{1,j}})$, $h_{2,k}(X_{2,k})$, and $h_{3,l}(X_{3,l})$ correspond to functions of one, two, and three parameters, respectively, and M_1 , M_2 , and M_3 are their total numbers. $x_{I_{1,j}}$ is a parameter indexed by $I_{1,j}$ (an integer ranging from 1 to D). $X_{2,k}$ and $X_{3,l}$ correspond to a parameter pair and a parameter group of three, respectively. We could include additive terms that are functions of four or more parameters, but such an implementation is found to be unimportant in our subsequent analysis, and hence not adopted. For several target variables, *sage* constructs their emulators separately following the general format of Equation 2.

We set each term in Equation 2 as a GP. Different from formal additive GP and the more commonly-used GP that is not additive, we specify the hyperparameters of the GPs in *sage* rather than estimating their values, and we only focus on the means of the GPs (μ^{GP}), and ignore the corresponding estimated uncertainty. Equation 2 can be rewritten as:

$$f(x_1, x_2, \dots, x_D) \approx \sum_{j=1}^{M_1} \mu_{1,j}^{GP}(x_{I_{1,j}}|\theta_1) + \sum_{k=1}^{M_2} \mu_{2,k}^{GP}(X_{2,k}|\theta_2) + \sum_{l=1}^{M_3} \mu_{3,l}^{GP}(X_{3,l}|\theta_3), \quad (3)$$

where $\mu_{1,j}^{GP}$, $\mu_{2,k}^{GP}$, and $\mu_{3,l}^{GP}$ correspond to the means of the one-, two-, and three-parameter GPs, respectively. θ_1 , θ_2 , and θ_3 are the fixed, specified GP hyperparameters.

We do not estimate the GP hyperparameters because we only focus on emulating the non-local, low-frequency relationship between the parameters and the output variable. Otherwise, the high-frequency variation, which could be overfitted by the emulator (due to the sparsity of the PPE), will likely be mistakenly emulated as signal rather than more appropriately treated as noise.

3.1.1. Sage Ansatz

Sage is designed to be additive for simplicity and interpretability. *Sage* selects the individual parameters, and then the parameter pairs and groups of three sequentially. There are alternative ways to formulate Equations 2 and 3. For

example, we can begin with parameter pairs first and ignore the individual parameters. We experimented with such possibilities, and they do not significantly improve the emulator performance. Additionally, emulating two or more variables first increases the likelihood of overfitting at the beginning of the iterations. This would degrade the emulator performance. We thus prioritize the variability that can be explained by individual parameters first. This design does not negate that all parameters somewhat interact with each other and consequently affect the value of the target variables. Instead, it assumes that such interaction is too complicated to capture given the sparsity of PPEs.

This parameter search order together with the pre-specified GP hyperparameters makes *sage* less sensitive to the particular (and true) ansatz of the target variable. For example, a function $f(x_1 \times x_2)$ would be decomposed as $g(x_1) + g(x_2) + g(x_1, x_2)$ by *sage*. This is not accurate, but the pre-specified GP hyperparameters will make the first two terms near a constant zero if the two parameters do not individually contribute to the variability.

3.1.2. Aspects of Gaussian Processes Used in This Method

The theoretical aspect of GPs is not critical to understanding *sage*. The role of the GPs in this method can be understood as a non-parametric interpolation method. See Rasmussen (2003) for a thorough and detailed introduction of GPs. We consider the below description sufficient in the context of the design of *sage*. The estimated mean of the GP is the weighted sum of the response (i.e., target variable in the present context) of the training data. The weights are determined by the covariance function which characterizes how similar or “close” a point of interest in the parameter space is relative to the others. We use the Matérn covariance function, the most common format of covariance function in this method (Rasmussen, 2003). The covariance function is defined by several hyperparameters. In particular, the *range* is a length scale that controls how “far” two points in the parameter space are considered to be correlated. With a greater *range*, the GP tends to model the low-frequency relationships existing in the data, because points that are “far” from the point of interest will be considered when making the prediction, leading to an overall smoother hyperplane. With a smaller *range*, more local and high-frequency fluctuation can be captured. The nugget to (signal) variance ratio can be understood as the noise to signal ratio. With these two hyperparameters specified, the GP hyperparameters of the covariance function are determined for a given set of training data (see Equation 11 of Gu et al. (2018)). We note here that the nugget (also known as noise level in some analyses) and (signal) variance are treated as two hyperparameters in some literature (Rasmussen, 2003). A simple example is given in Appendix A to illustrate how the *range* and nugget-to-variance ratio affect the emulated mean of a GP. *sage* is built from the R package “RobustGaSP” (Gu et al., 2018), but it can be easily written in python and based on other GP libraries due to its simplicity.

3.2. Defining the Workflow and Diagnostics That Enhance Interpretability

With the format of the emulator determined and the GP hyperparameters pre-specified, we must determine what parameters and parameter groups that $x_{1,j}$, $X_{2,k}$, and $X_{3,l}$ correspond to in Equation 3 and the numbers of the additive terms (i.e., M_1 , M_2 , and M_3). Determination of the parameters and parameter groups can be visualized with the flow chart outlined in Figure 1. It can be generally viewed as a stepwise forward selection method (see more details on this topic in Draper and Smith (1998)). A similar approach of method (parameter) selection has been adopted in the emulator of Williamson et al. (2013), D. Sexton et al. (2019). Alternatives to forward stepwise regression methods exist, such as best subset and lasso regression (e.g., Hastie et al. (2017)). The most distinct difference for *sage* is its criterion to select the effective parameter interaction (Equation 4) which will be introduced later. We note here that the code to implement *sage* is fully automated, but it is introduced in detail below with the hope that certain aspects of it can inform or be improved from future studies.

sage begins with initial guesses on M_1 , M_2 , and M_3 . Given D parameters, setting the initial guesses as $M_1 = \text{int}(D \times 3/4)$, $M_2 = \text{int}(D/3)$, and $M_3 = \text{int}(D/4)$, where int denotes taking the integer part of a number, are sufficient given our experience working with the two PPEs. *sage* then uses 80% of the training data to select M_1 single parameters, M_2 parameter pairs, and M_3 parameter groups of three (how the parameters are selected are given below). It then applies the emulator trained on 80% of the training data to the rest training data and evaluates its performance. As *sage* is additive, the evaluation against this 20% of the training data should lead to gradual decreased RMSE as more terms are added in the prediction. Some additive terms might lead to degraded prediction (i.e., they add noise to the data) which are excluded by *sage*. This exclusion automatically updates M_1 , M_2 , and M_3 (Figure 2a).

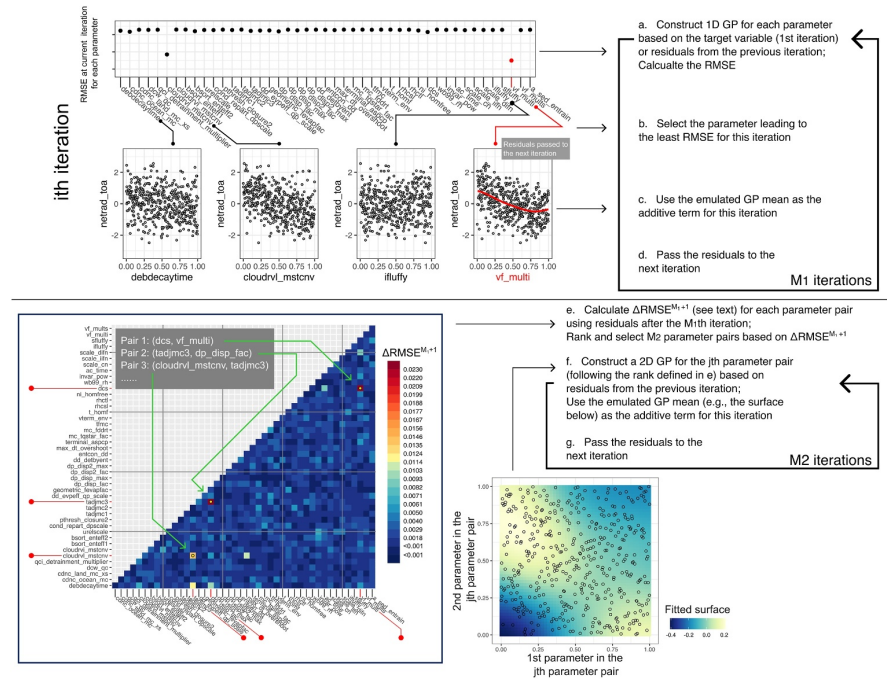


Figure 1. Workflow of *sage* for individual parameters and parameter pairs. The variable *netrad_toa* in the ModelE3 PPE Default Test is used as an example here.

The criterion for excluding a term or not can be specified by the user. The default is zero, but can be changed to a positive value if a slight degraded performance is considered acceptable. As presented in Figure 2a, the points marked with crosses do not lead to a decrease in the RMSE when applied to the 20% of the training data, and are thus excluded. The above process selects a sequence of parameters and parameter groups, and provides an emulator, although note that the emulator is trained only on 80% of the data. As the last step, *sage* trains a final emulator based on the *complete* data set using the updated parameters and parameter groups from the previous step (which excludes degrading parameters and parameter groups). It is still constructed sequentially. Examples of the method performance using the originally selected and updated parameters and parameter groups are given in Figures 2b and 2c, respectively.

3.2.1. Determining the Parameters and Parameter Groups

sage focuses on determining single parameters ($x_{i,j}$) first, which is done iteratively and sequentially. In the first iteration, it aims at emulating the target variable, and in the following ones, it emulates residuals from the previous iteration. In each iteration, it (a) constructs a one-parameter GP for each of the D parameters with the same pre-specified hyperparameters, (b) calculates and examines the RMSE from the D GPs based on their mean predictions, and picks the parameter corresponding to the lowest RMSE as the selected parameter in this iteration, (c) uses this selected emulated GP mean as the additive term for this iteration and (d) passes the residuals from the fitting (based on the GP mean) to the next iteration. The above four steps correspond to Figures 1a–1d in the flow chart. We allow a single parameter to be selected more than once in case that parameters could contribute to the fitting multiple times. As the pre-specified GP *range* hyperparameters is large, the second time a parameter is selected would not lead to overfitting to the high-frequency variability. Our ablation experiments given in Appendix B suggest that the second or more times a parameter is selected in most cases slightly improves the emulator performance. This feature is hence enabled. We acknowledge this heuristic aspect of *sage*. To prevent a parameter from being reselected, users could uncomment Line 37 in function *f_seq_gaus_1d* in the Zenodo repository (version: v0; (Yang et al., 2025)).

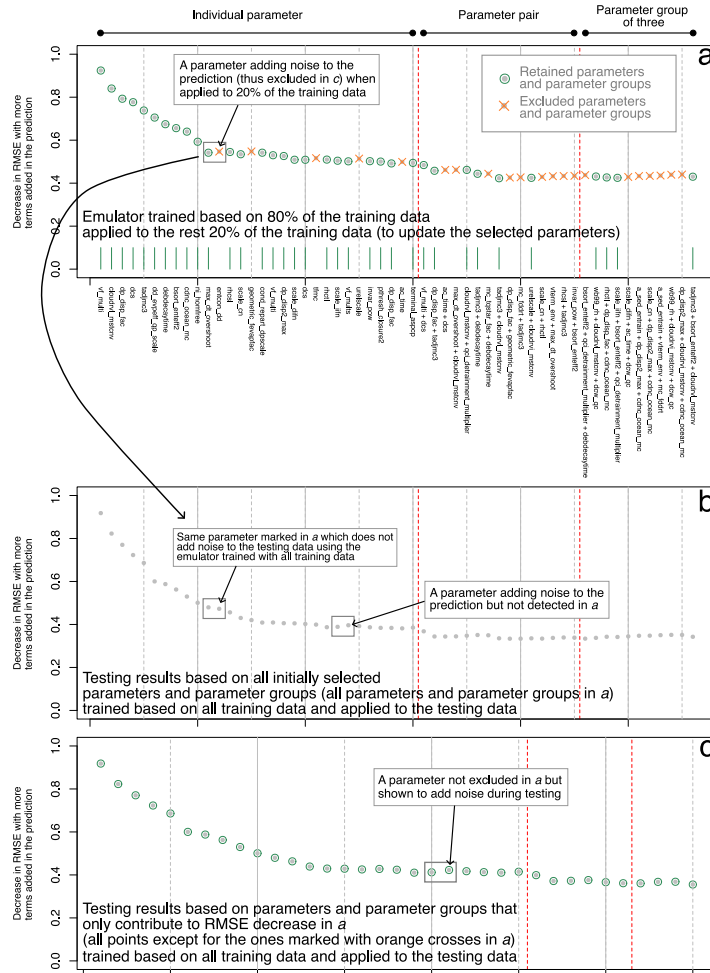


Figure 2. Example showing how *sage* works using the example of *netrad_toa* from the ModelE3 PPE. The three figures are the output of *sage*. In all three figures, the y-axis corresponds to the RMSE after more terms are included in the prediction. Its value generally decreases as more additive terms (x-axis, the corresponding parameters and parameter groups) are included in the prediction. The x-axes in panels (b, c) are not labeled for simplicity. (a): The parameters are selected using 80% of the training data. The corresponding emulator is applied to the rest 20% of the training data (to update the selected parameters). The points circled in green and marked by the green bars contribute to the decrease of the RMSE, and are hence retained as the final parameters and parameter groups. The points marked by crosses lead to additional noise in the prediction, and the corresponding parameters are hence excluded. Two separate emulators using the originally selected (all parameters in a) and final (those that are not marked by the orange crosses in a) parameters and parameter groups, trained based on the complete data set, are applied to the same testing data in b and c, respectively.

After the M_1 iterations are complete for the single parameters, *sage* moves on to determine the parameter pairs ($X_{2,k}$). We will have $\binom{D}{2}$ parameter pairs. For *each* of the parameter pairs x_p and x_q , we construct three GPs for the residuals after the M_1 th iteration (i.e., after the individual parameters are all selected): two one-parameter GPs for the two parameters respectively, and another two-parameter GP for (x_p, x_q) . The three GPs will lead to three RMSEs, denoted as $RMSE^{M_1+1}(p)$, $RMSE^{M_1+1}(q)$, and $RMSE^{M_1+1}(p, q)$, respectively. Denoting the $RMSE^{M_1}$ as the RMSE after the M_1 th iteration (i.e., the RMSE of the prediction as the sum of M_1 additive terms), we use $\Delta RMSE^{M_1+1}(p, q)$ to evaluate and rank the additional improvement in the fitting if we consider the interaction of parameters pairs:

$$\Delta RMSE^{M_1+1}(p, q) = (RMSE^{M_1} - RMSE^{M_1+1}(p, q)) - (2 * RMSE^{M_1} - RMSE^{M_1+1}(p) - RMSE^{M_1+1}(q)). \quad (4)$$

The first two terms on the RHS (right hand side) reflect how much variability is explained by using the GP that considers x_p and x_q jointly, and the last three terms correspond to the variability explained by considering x_p and x_q independently. Greater $\Delta RMSE^{M_1+1}(p, q)$ indicates greater benefit in emulating the two parameters jointly.

Since the impact of individual parameters is considered in previous iterations, the last three terms on the right hand side of Equation 4 would be similarly low for any parameter pair. They would not greatly affect the value of $\Delta RMSE^{M_1+1}(p, q)$. We keep these three terms in Equation 4 to highlight that this measure denotes the *additional* benefit of emulating two parameters jointly. With Equation 4, we can quickly detect parameter interaction without the need to implement the iterations for single parameters (i.e., we could skip the iterations for single parameters). If the last three terms in Equation 4 are considered redundant, they can be modified in the code by users (i.e., remove the last three terms at Line 20 in function *seq_gaus_2d* and remove the last four terms at Line 21 in function *seq_gaus_3d* for parameter groups of three in the Zenodo repository Yang et al. (2025); version: v0). The calculation of Equation 4 for all parameter pairs corresponds to step e in Figure 1.

We rank and select M_2 parameter pairs with the greatest $\Delta RMSE^{M_1+1}(p, q)$. *sage* then sequentially emulates the residuals from the previous iteration, and passes the residuals from the fitting to the next iteration using each of the selected and ranked parameter pairs as the GP inputs (Figures 1f and 1g). A similar procedure is then implemented for parameter groups of three after a total of $M_2 + M_1$ iterations. The only difference is that we modify Equation 4 for parameter groups of three such that it denotes the additional benefit of emulating three parameters jointly compared to emulating them independently.

We could select parameter pairs and parameter groups of three through iterations, as is done for the individual parameters. This is easy to implement in the code, although it would be computationally time-consuming, and thus is not adopted in the current version of the code. This means that a parameter pair or parameter group of three will not be selected for more than once. Namely, we allow the individual parameters to be selected for more than once based on the ablation study given in Appendix B. For parameter groups, this is not allowed for computational considerations. The complete training process leads to two products: (a) an ordered sequence of several parameters, parameter pairs, and parameter groups of three and (b) the trained emulator for the target variable.

3.3. Including the Relationship Between Outputs

The level of difficulty in emulating different target variables may vary. It is possible that an easier-to-emulate variable A could help emulate variable B that is more difficult to emulate. We can explore whether such relationships exist by including output variables that are easier to estimate as parameters, and apply *sage* focusing on emulating the other variables. We pursue this in Section 6.3.

4. Emulator Evaluation

As mentioned, we apply *sage* to the ModelE3 and CAM6 PPEs independently. We evaluate its performance using randomly sampled ensemble members for training and validation and with varied GP hyperparameters. Its performance is compared with a NN emulator. How each parameter and parameter group contributes to the emulation is a key component of our workflow and analyses.

The NN used in this work is the ensemble average of four fully connected NNs with varying numbers of hidden layers (3–5), nodes (32–256), and activation functions (ReLU and leaky ReLU). The NN emulates all variables all at once. We choose to compare against a NN emulator over the conventional, non-additive GP since the NN slightly outperforms the latter (Appendix C).

4.1. Default and Random Sampling Tests

We first define Default Tests for the two PPEs, respectively, for better visualization and easier examination of subsequent results. For the ModelE3 PPE, we use the first 1–451, 552–631, and 652–731 (a total of 611) ensemble members in the complete data set to train the emulator, and set aside 140 for validation. For the CAM6 PPE, the first 210 ensemble members are selected for training, with 52 retained for validation. Note that the last 200 ensemble members (552–751) in the ModelE3 PPE are clustered around a region in the parameter space with

Table 2
Specified GP Hyperparameters

Set name	Range for 1-D GP	Range for 2-D GP	Range for 3-D GP	Nugget to variance ratio
Default Test	0.60	$\sqrt{0.5^2 + 0.5^2}$	$\sqrt{0.4^2 + 0.4^2 + 0.4^2}$	2.00
Set 1	0.60	$\sqrt{0.5^2 + 0.5^2}$	$\sqrt{0.4^2 + 0.4^2 + 0.4^2}$	4.00
Set 2	0.60	$\sqrt{0.5^2 + 0.5^2}$	$\sqrt{0.4^2 + 0.4^2 + 0.4^2}$	1.00
Set 3	0.80	$\sqrt{0.6^2 + 0.6^2}$	$\sqrt{0.4^2 + 0.4^2 + 0.4^2}$	2.00
Set 4	1.00	$\sqrt{0.8^2 + 0.8^2}$	$\sqrt{0.6^2 + 0.6^2 + 0.6^2}$	2.00
Set 5	0.50	$\sqrt{0.4^2 + 0.4^2}$	$\sqrt{0.3^2 + 0.3^2 + 0.3^2}$	2.00

greater likelihood of approximating the observations. We take 160 samples from them to ensure that the training data are representative of the complete PPE. Both Default Tests retain 18–19% of the ensemble members for validation. In addition to the Default Tests, we also randomly draw ensemble members for training and validation from both PPEs 10 times (another 10 random sampling tests). The numbers of training and validation members are identical to those in the respective Default Tests. These random sampling tests are implemented to detect how the ensemble members used for training and validation affect the emulator performance. Throughout this work, the same 11 sets (a Default Test plus 10 random sampling tests) of training and validation data sets are used for each PPE for consistency unless specified otherwise. In working with the ModelE3 PPE, the parameter sequences are based on the training samples that are within the first 551 ensemble members such that the parameter sequences are obtained based on data that are evenly distributed in the parameter space. These parameter sequences are then coupled with the complete training data to train the final emulators.

We apply both *sage* and NN to the 11 sets to evaluate and compare their performance for each PPE. For both methods, different emulators are trained in each test. The GP hyperparameters are fixed and provided in Table 2. These will be varied later, as explained. All parameters are normalized to range from zero to one, and target variables are standardized to be zero-mean with a standard deviation of one. The hyperparameters *ranges* specified in Table 2 ensure that they are at least 30% (i.e., *range* for 3-D GP in Set 5; $\sqrt{0.3^2 + 0.3^2 + 0.3^2} / \sqrt{1^2 + 1^2 + 1^2}$) of the maximum distance ($\sqrt{1^2 + 1^2 + 1^2}$) of the parameter spaces, which ensures that we emulate the non-local, low-frequency relationship between the target variables and parameters.

5. Emulator Performance

The resultant R-squares of the two PPEs and their variability from using the two methods are compared in Figures 3a and 4a. The performance of the two methods is comparable for the ModelE3 PPE with the NN exhibiting better performance for the variables that are relatively difficult to emulate (i.e., those with lower R-square for both methods). Among the 33 target variables, *sage* outperforms the NN in emulating variables *TIWP* (Total Ice Water Path; highlighted in Figure 3a) and *tcc_sc* (stratocumulus cloud cover) in both the Default Test R-squares (absolute R-square differences between the two methods: 0.21 and 0.07) and its variability from random sampling. Twenty variables have their R-square differences between the two methods within the range of -0.05 and 0.05 in the Default Test. The corresponding R-square variability is also comparable. For the other 11 variables, marked green in Figure 3a, the R-square differences between the two methods exceeds 0.05 (two variables below 0.10; five in the range of 0.10–0.20; and four in the range of 0.20–0.32). The NN outperforms *sage* in emulating these.

Both methods exhibit degraded performance in emulating the CAM6 PPE relative to the ModelE3 PPE. For CAM6, *sage* has an overall improved performance relative to the NN (Figure 4a), and both are characterized by large variability in R-square in the random sampling tests. Among the 21 variables emulated, there are 6 (there is a pair of variables overlapping in Figure 4a) variables that both methods emulate relatively and comparatively well: their R-squares in the Default Test are above 0.63 using both methods with similar variability from random sampling. *sage* outperforms the NN in emulating another three variables with their corresponding R-squares being 0.58, 0.67, and 0.70 using the former in the Default Test. Their R-square variability is comparable or slightly better when *sage* is used. The NN outperforms *sage* in emulating the variable *T_925hpa* (with R-square

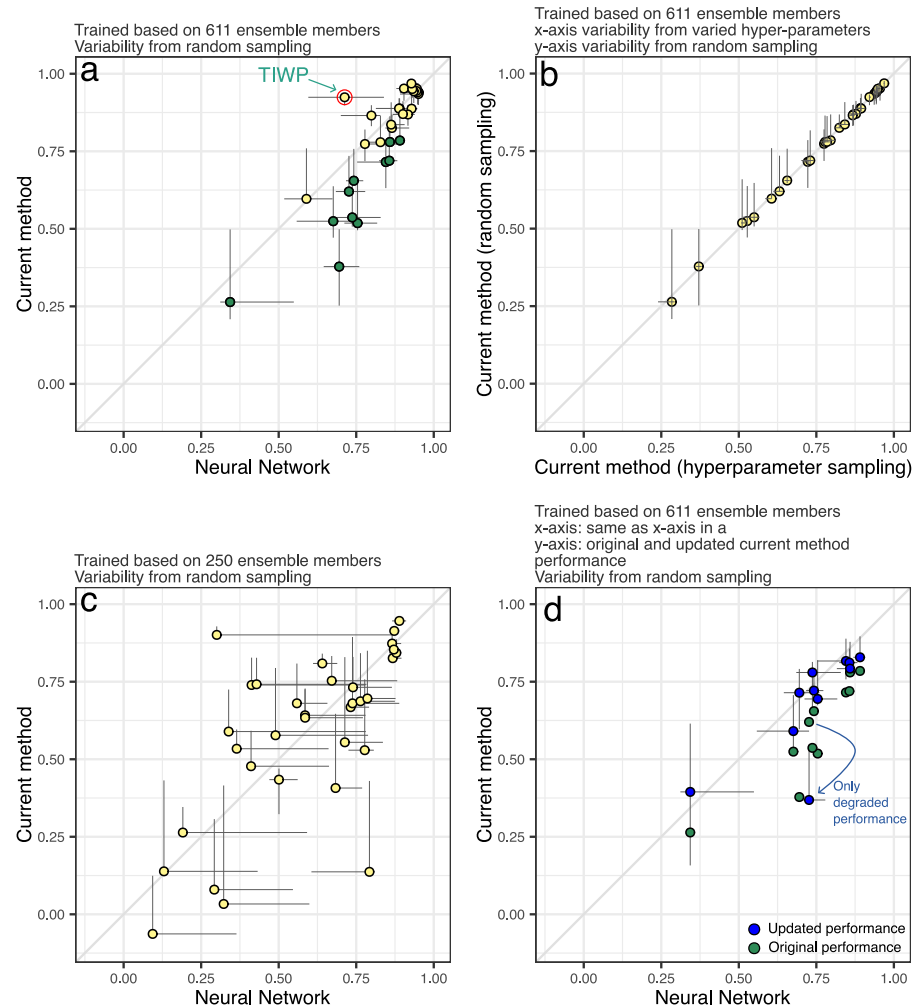


Figure 3. R-squares and their variability from different tests with the ModelE3 PPE using *sage* and the NN. (a, c): Comparison between *sage* and NN given 611 and 250 ensemble members for training. Green points in panel (a) correspond to variables that NN is better at emulating. The points are marked with different colors in a as the green points will be used for additional analysis (shown in panel d). Since no additional analysis is done for points in panel (c), we do not label them with different colors. (b): R-square variability using *sage* from random sampling (y-axis) and from varied GP hyperparameters (x-axis) with 611 ensemble members for training. (d): updated performance (blue points) of *sage* (see text for details) compared with its original performance and NN trained with 611 ensemble members for the variables that the NN is better at predicting (green points in panel a). All points are plotted based on the Default Test (a, b, d) or the test that uses the first 250 ensemble members (c) for training. All vertical and horizontal bars denote variability from random sampling except for the horizontal bars in b which denote the variability from varied GP hyperparameters.

greater than 0.5 using Neural Network and below 0.5 using *sage* in the Default Test in Figure 4a). Neither *sage* nor the NN performs well in emulating the rest variables. They have either low or even negative R-squares in the Default Test or extreme R-square variability from the random sampling tests. (The negative R-squares arise when the emulator introduces additional bias and noise to the prediction, e.g., for a set of points on the line $y = -x$, an emulator in the form of $y_{emu} = -x$ could lead to negative R-squares). Another three variables have their R-squares or variability outside the extent of Figure 4a and are not shown.

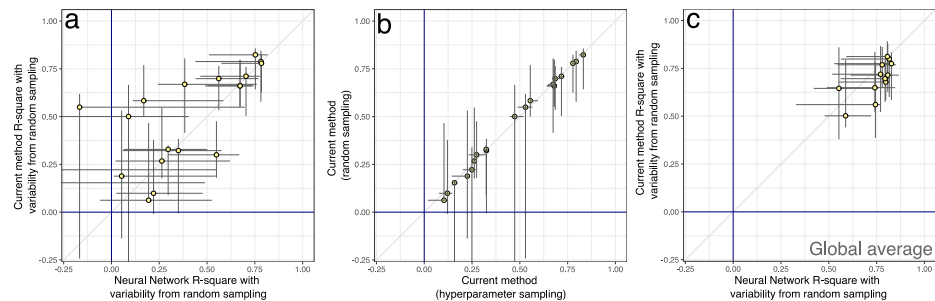


Figure 4. R-squares and their variability with the CAM6 PPE using *sage* and NN. (a): R-squares from using the two methods. The points are plotted based on the Default Test results, and the bars denote the variability from random sampling; (b): R-squares of *sage* with the variability from random sampling (vertical bars) and from varied GP hyperparameters (horizontal bars). Points in (a, b) are plotted based on the Default Test. More points are shown in panel (b) than (a) because some variables have their R-squares outside the extent in using NN; (c): the same as (b) except that the target variables are all global averages. These variables are marked in Table 1.

Considering the two PPEs, the two methods perform differently not only because the models, parameter ranges and samples are different, but because some target variables in both PPEs are skill score metrics rather than model output climatologies (Table 1). We illustrate this in an additional analysis of the CAM6 PPE in Section 7.2.

5.1. Varied Hyperparameters

Another five experiments are performed using the training and validation data sets in the Default Tests of the two PPEs. Different GP hyperparameters defined in Table 2 (Sets 1–5) are used here, and the same parameter sequences as the Default Tests are used. The corresponding R-square variability is compared with that from the random sampling tests using *sage* (Figures 3b and 4b). The R-square variability from the varied hyperparameters is much smaller than that from the random sampling tests for both PPEs, showing that the GP hyperparameters having very limited impact on the emulator performance.

5.2. Contributions From Individual Additive Terms

Since the emulator prediction is additive, we are able to examine how each parameter or parameter group contributes to the normalized RMSE decrease (normalized for the testing data) during the training or validation. The latter is preferred to ensure robustness. We call the decrease in the RMSE “explained variability” in the following text. Examples for target variables *olr* (outgoing long wave radiation) and *TIWP* in the ModelE3 PPE Default Test are presented in Figure 5. (We note here that *sage* automatically outputs the explained variability from each parameter and parameter group.)

We calculate the explained variability for all variables based on the Default Tests of the two PPEs (Figure 6). For parameter groups, only those with explained variability greater than 0.02 are presented in Figure 6 to avoid redundancy in the visualization. The crosses in Figure 6 denote the individual parameters that contribute negatively to the prediction (negative explained variability). They are selected during the training process but shown to degrade the prediction in the Default Tests. Their total impact on the emulator performance is different for the two PPEs, which will be examined in Section 5.3.

The most noticeable features shown in Figure 6 are briefly summarized here. For the ModelE3 PPE, the parameter *dcs* (threshold diameter for autoconversion of cloud ice to snow) contributes the greatest to the explained variability of most target variables. Parameter *vf_multi* (multiplier on coefficient for cloud ice fall speed) helps greatly with the emulation prediction, but the explained variability from it is smaller relative to *dcs*. As a pair, the two parameters also contribute jointly to the emulation of several target variables (e.g., variable *pwv*, column water vapor).

For the CAM6 PPE, there are also two parameters, namely *micro_mg_autocon_lwp_exp* (KK2000 LWP exponent, see more details in Gettelman and Morrison (2015)) and *micro_mg_vtrmi_factor* (ice fall speed scaling), that dominantly contribute to the emulator prediction as individual parameters. Parameter groups do contribute to

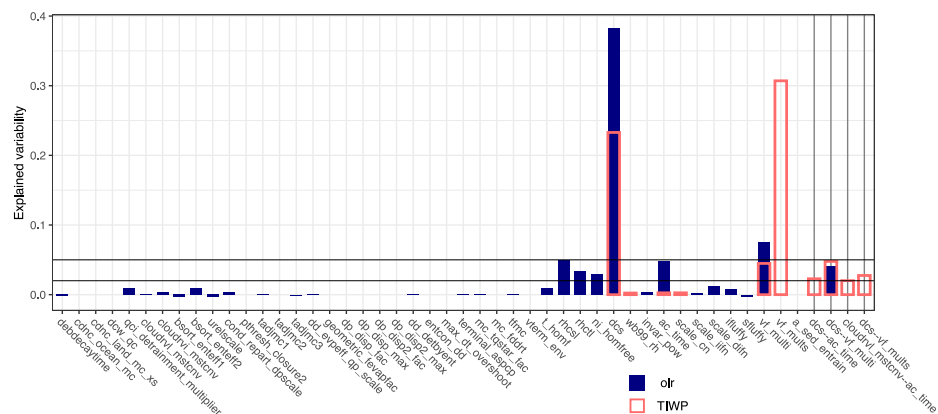


Figure 5. Explained variability for all individual parameters plus parameter groups with explained variability greater than 0.02 (the lower horizontal line) for variables *olr* and *TIWP*. Calculated based on the ModelE3 PPE Default Test.

the prediction of some variables but with explained variability much smaller compared to the parameter groups in the ModelE3 PPE.

5.2.1. The Unexpected Success in Emulating *TIWP* for the ModelE3 PPE and Its Implication

sage greatly outperforms the NN in emulating the ModelE3 PPE *TIWP* (highlighted in Figure 3a). This markedly improved emulation was unexpected. Successful emulation is dominated by roughly 6 parameters (the parameters that are crossed by the horizontal line in Figure 6a), suggesting that this variable is not difficult to emulate by *sage*. Why such an improvement? Parameter *vf_mults* (multiplier on coefficient for snow fall speed) contributes the most to the emulation of *TIWP*, and it contributes very little to all other variables. The relationship between *TIWP* and *vf_mults* is effectively independent from the remaining target variables and parameters. Because of this independence, we suspect that for a NN with 33 variables to emulate and with only 611 ensemble members to train from, it is unable to isolate the single parameter-variable relationship (i.e., *TIWP* and *vf_mults*) well compared with the multi-variable-parameter relationship.

We delve deeper by constructing another two NNs focusing solely on “*TIWP*.” The first one utilizes all 45 parameters as inputs and the second one utilizes the six parameters revealed as most important for *TIWP* by *sage* (i.e., the parameters that are crossed by the horizontal line in Figure 6a). The R-square, based on training and testing ensemble members from the Default Test, from the first NN varies greatly, ranging from 0.65 to 0.85 (in most cases around 0.69–0.75) in different training sessions; the second NN leads to R-square with limited variability, ranging from 0.86 to 0.93, a range comparable to the performance of *sage*. The above results are invariant to different NN structures and setups (e.g., numbers of hidden layers, activation function, epochs).

The above experiments reveal important implications on emulator design for climate model PPEs. They show that the performance of the NN could be improved if two or more emulators are constructed to emulate separate groups of target variables. They are also consistent with our hypothesis that the better performance of *sage* on *TIWP* is related to its stand-alone relationship with *vf_mults*, as once given the correct parameters selected by *sage*, the performance of NN improves greatly. The above arguments are important as similar scenarios might exist for other PPEs or other emulator methods. The above arguments cannot be directly derived from a NN, showcasing the importance of analysis on climate model PPEs, which should be distinguished from analyses of surrogate PPEs based on emulators that might produce more error-laden outputs for some variables. We note here that causal methods together with the inclusion of regularization terms (e.g., LASSO) in the loss function of machine learning algorithms have been proposed to improve climate models and projects (e.g., Iglesias-Suarez et al. (2024)). What such methods and *sage* share in common is the parameter selection procedure. However, as noted below, our finding that individually less important parameters could be cumulatively important suggests that the parameter selection needs to be implemented with caution.

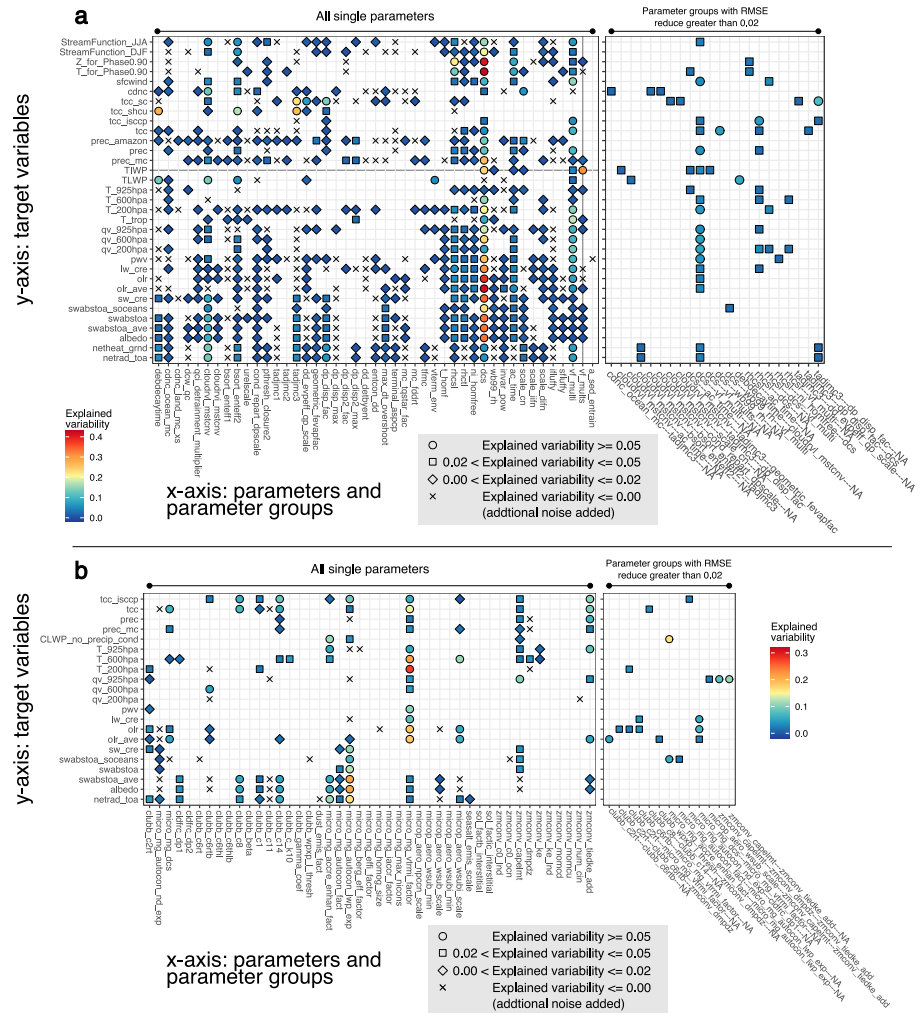


Figure 6. Explained variability from all individual parameters plus parameter groups with explained variability greater than 0.02 in the Default Tests of the ModelE3 Perturbed Parameter Ensemble (PPE) (a) and CAM6 PPE (b). The crosses denote the parameters that introduce additional noise in the prediction.

5.3. Cumulative Contributions From Additive Terms

We group and sum the explained variability from each additive term based on the number of parameters it corresponds to (i.e., 1, 2, and 3) and the value of the explained variability in the Default Tests of the two PPEs. We choose 0.05 and 0.02 as the thresholds for the latter criterion. The group that contributes to less than 0.02 RMSE includes the terms with negative contribution (i.e., crosses in Figure 6). The classification based on the value of the explained variability is not done for parameter groups of three as their total contribution is already small in most cases. The results are presented in Figure 7 for the two PPEs.

For the ModelE3 PPE (Figure 7a), the variability in the total (summed) explained variability from random sampling is relatively small for most target variables. The comparable performance between *sage*, a simple and additive emulator, and NN, a method that is potentially capable of capturing highly non-linear and high-frequency variation in the training data, allows us to draw the following points with confidence (see Appendix D, which further confirms the robustness of the points below). These points are listed with brevity but their

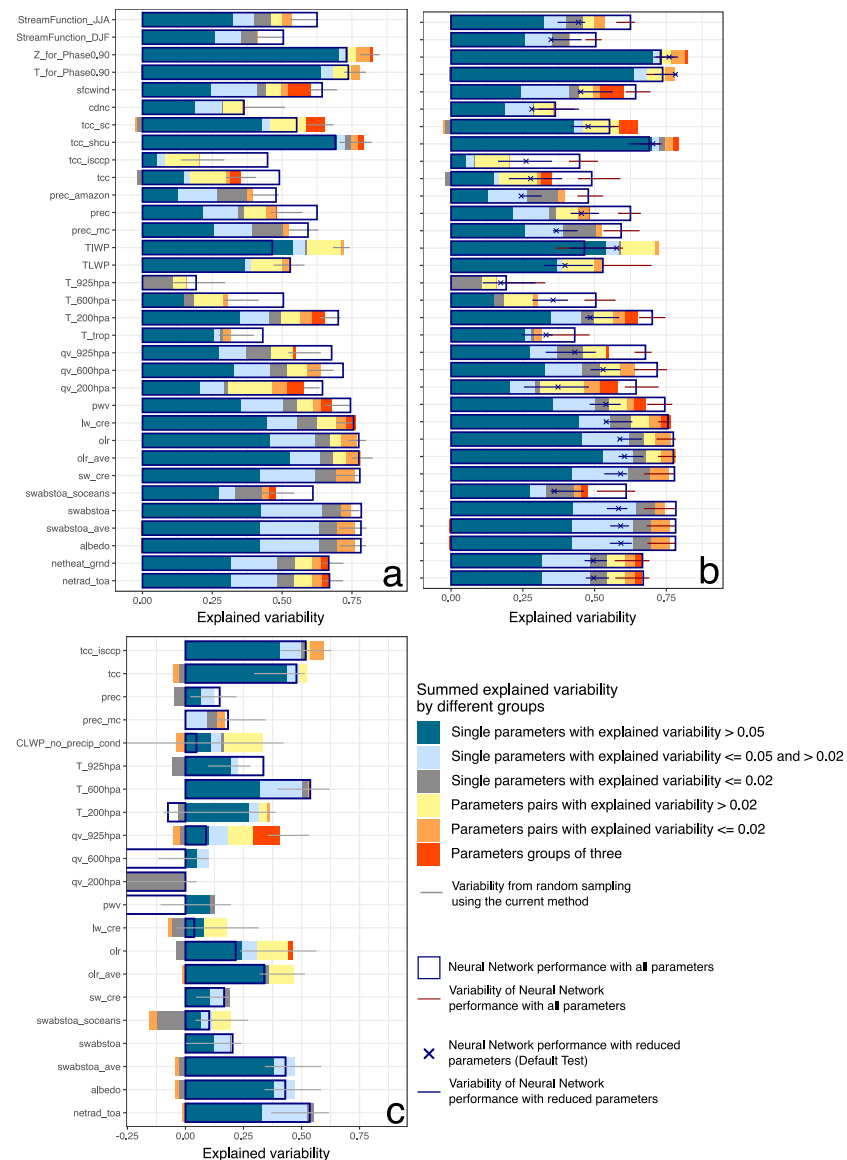


Figure 7. Explained variability (x-axis) grouped and summed based on different criteria in the Default Tests of the ModelE3 Perturbed Parameter Ensemble (PPE) (a, b) and CAM6 PPE (c) denoted as different colored bars. The total variability explained from the NN is marked using the dark blue empty boxes. The total variability from random sampling using *sage* is plotted as horizontal lines in panels (a, c). (b) Is identical to a except for the following: the crosses denote the total explained variability from a NN trained on 13 selected sensitive parameters (see text for more details). It is based on the Default Test training and testing data. The red and blue bars in b denote the variability of the total variability explained using the NN due to random sampling based on all and the 13 sensitive parameters, respectively.

importance should not be neglected: (a) most of the explained variability is from the contributions of individual parameters; (b) surprisingly, the cumulative (as *sage* is additive) effect of parameters that individually contribute to less than 0.05 of the explained variability is not negligible (which we separate into two categories: between 0.05 and 0.02, and below 0.02); (c) The explained variability from parameter groups is relatively small, but cannot be neglected (e.g., *TIWP*); (d) the contribution from parameter pairs is generally greater than

that from parameter groups of three. In most cases (with two exceptions with negative gray bars in Figure 7a), the group with individual explained variability less than 0.02 improves the emulator performance, confirming that the overall impact of the additive terms with negative explained variability (crosses marked in Figure 6a) is limited for the ModelE3 PPE.

The second point listed above deserves additional attention, as it is related to how we should interpret the sensitivity of climate model parameters. This can be seen with the variable *olr* in Figure 5. Five terms (four as individual parameters and one as parameter pair) have their explained variability in the range of 0.02–0.05 with a total sum of more than 0.15, an amount that should not be neglected in the emulator construction or sensitivity analysis. The cumulative effect of these individually less important parameters will be further studied with NN in Section 6.2.

Analysis of the ModelE3 PPE also shows that the parameter importance and sensitivity can be underestimated if we examine it without considering parameter interaction. We use variable *TIWP* as an example to illustrate this (Figure 5). For this variable, parameter *dcs* is not only important individually, it also pairs with another three parameters. The summed explained variability from these three pairs is greater than 0.09 (two above 0.02 and one around 0.05). This is not negligible as the total explained variability for this variable is 0.72 in the Default Test.

Results (Figure 7c) for the CAM6 PPE show that most effective and robust explained variability is from individual parameters that contribute to more than 0.05 of the explained variability. The cumulative effect of other individual parameters and parameter groups is in most cases relatively small or comparable to the total explained variability from the random sampling tests. There are not that many parameters or parameter groups that contribute to the degradation of the fitting (i.e., fewer crosses in Figure 6b; compared to the case with the ModelE3 PPE), but their individual negative impact is large (i.e., the wide negative bars in Figure 7c). The performance of both methods is low, suggesting the greater difficulty in emulating the CAM6 PPE. The variability in the total explained variability due to random sampling is relatively large, preventing us from making further interpretations.

As mentioned earlier, we allow *sage* to reselect individual parameters. Ablation analysis (Appendix B) suggests that this in most cases slightly improves the method performance, and this feature is kept in the current version of *sage*.

Since the performance of *sage* varies greatly for the two PPEs, they are analyzed separately below. For the ModelE3 PPE, we focus on what insights can be obtained from working with *sage* and comparing it with NN. For the CAM6 PPE, we focus on measures to improve the emulator performance.

6. Analysis on the ModelE3 PPE

We focus on three aspects of the ModelE3 PPE: how the performance of *sage* or NN changes in response to reduced (a) number of training ensemble members and (b) number of parameters, and (c) whether the variables that *sage* emulates well could inform the emulation of those that are difficult to predict.

6.1. Reduced Number of Ensemble Members

We used 250 ModelE ensemble members to train *sage* and the NN, and evaluate their performance using the remaining ensemble members. The first 250 ensemble members plus another 10 randomly sampled data sets of same size are used for training and the rest for validation, which amounts to 11 tests. The GP hyperparameters of the Default Test are also used here. The R-square and its variability from the 11 tests using *sage* and NN are presented in Figure 3c. The points are evenly split by the one-to-one line, suggesting comparable performance from the two methods. For the variables that either method is better at emulating, the variability from random sampling is much smaller compared to the other method.

We select variables *olr_ave* and *StreamFunction_DJF* for more detailed analysis. They are chosen since the *sage* performance on the two variables are greatly different (comparable performance for the former using the two methods and lower performance for the latter using *sage*; Figure 7). We randomly select 100, 200, 300, and 400 ensemble members from the first 451 ensemble members for training and then apply the trained emulators to the 452–551 ensemble members for validation and the calculation of the explained variability. This process is repeated five times (but the target is always the 452–551 ensemble members) for each number of ensemble

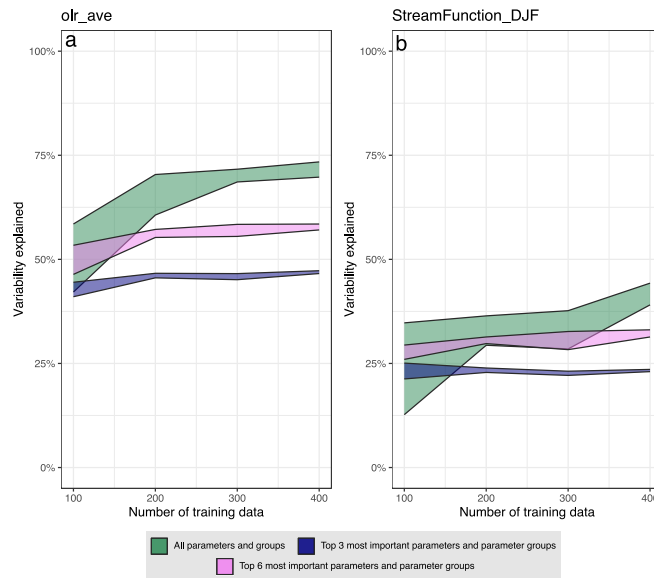


Figure 8. Total variability explained by all, the first three, and the first six most important parameters and parameter groups for the variables *olr_ave* (a) and *StreamFunction_DJF* (b) for the ModelE3 PPE given different numbers of ensemble members for training. The variability is calculated based on applying emulators trained on randomly sampled ensemble members (from the first 451 ensemble members) to the same 452–551 ensemble members.

members to produce ranges of variability explained. We do not consider the other ensemble members (552–751 ensemble members) as they are not randomly distributed in the parameter space, which might be a factor affecting the variability explained by *sage*. The total variability together with the variability explained by the first three and six most important parameters and parameter groups are presented in Figure 8.

For *olr_ave*, the total explained variability increases with the number of training ensemble members, and plateaued after training on 300 ensemble members. For this variable, if we already have 300 ensemble members, the expected emulator improvement upon sampling another 100 ensemble members is negligible. In contrast for *StreamFunction_DJF*, as we increase from 300 to 400 members, the variability explained by all parameters and parameter groups increases by nearly 15%. The above results suggest that the emulator performance can be demonstrably improved by increasing the number of training ensemble members, although this conclusion is output-variable specific.

What the two experiments share in common is that the variabilities explained by the first three and six most important terms do not change greatly with the number of training ensemble members. This is not surprising, given that most of the variability explained tends to be from individual parameters (Figure 6), and such independent and individual effects of these parameters do not need that many (in this case, 300 or 400) ensemble members for the emulators to capture.

Quantitative analyses such as those presented in this section suggest an iterative approach to sampling, weighing the quantitative increase in emulator performance against the cost of additional samples.

6.2. Reduced Parameter Space

As shown earlier, parameters that do not contribute greatly to the emulator performance individually (i.e., those with explained variability below 0.05) could together have a non-negligible and cumulative impact on the emulator performance (Figure 7a). This is demonstrated using *sage*, but might not be valid for other emulators. To test its universality, we keep the 13 sensitive parameters that individually contribute to more than 0.05 of the explained variability to at least one target variable in the ModelE3 PPE Default Test, exclude the other

parameters, and implement the Default Test plus the corresponding 10 random sampling tests using the NN. The same procedure is not performed for *sage* as it is additive, and the results are hence already presented in Figure 7a.

The total explained variability and its variability using NN with the complete and reduced parameter spaces are compared in Figure 7b. For a variable whose explained variability from individual parameters is low but the summed explained variability is large (i.e., those with long light blue and gray bars in Figures 7a and 7b; e.g., *albedo*), the degradation of the NN performance is consistently large with the reduced parameter space. The NN performance remains unchanged for the variables that are dominated by individually more important parameters (i.e., those with long dark blue bars and short light blue and gray bars in Figures 7a and 7b; e.g., *tcc_shcu*, shallow cumulus cloud cover). The cumulative effect of individually less important parameters is thus important to the NN as well, suggesting that its importance is not method-specific.

6.3. Relationship Between Variables

The *TIWP* emulation case discussed earlier suggests it is not necessarily optimal to emulate all variables simultaneously. At the same time, it is intuitive that some target variables can be related to one another (arising from underlying physics, and the redundancy in target variable selection), and leveraging such relationships could improve emulator performance.

We explore this in the ModelE3 PPE, focusing on the variables that are difficult to emulate by *sage* (i.e., the points marked green in Figure 3a). We select the variables with R-square greater than 0.85 in the Default Test using *sage* as the easy-to-emulate variables, with the expectation that they could help emulation of the “difficult” variables.

During the training, we include the “easy” variables as parameters, and obtain a parameter sequence for each “difficult” variable based on the training data set. The parameter sequences therefore have the original parameters and the “easy” variables. The “easy” variables are provided by the original emulators (i.e., the ones trained using the original 45 parameters). The 11 training data sets (i.e., the Default Test and the corresponding 10 random sampling tests) with 611 ensemble members are used for training here.

The original (i.e., trained with the original parameters) and updated (i.e., trained with the inclusion of the “easy” variables as parameters) R-squares of the “difficult” variables are plotted and compared in Figure 3d. The R-squares increase for all “difficult” target variables with one exception. There are some variables with significant increase in the updated R-squares in all 11 tests, suggesting robust improvement in the emulator performance. The experiment results shown in Figure 3d suggest that target variables that are related should be grouped together to optimize the emulator performance.

The improved performance is related to how the target variable is defined. In the original emulator experiments, some target variables are defined to be model penalty scores (e.g., output of 1 and -1 judged relative to an observation of 0 is equivalently 1 to the model score), which prohibits detectable relationships that could help with the emulator performance. This increases the difficulty in emulating such model scores if the emulator is not informed of the relationship between target variables, that is, how the original emulators are trained in the Default Test. A simple example is given in Appendix E to illustrate this. The explained variability from the additive terms for the “difficult” variables in this experiment together with biplots of selected parameters and variables are given in Appendix F. It shows that most of the variability is explained by those “easy” variables used as parameters here. This improved performance is not presented as the main results in this work because a subjective PPE-specific decision needs to be made on the definition of “easy” and “difficult” variables.

7. Improving the Performance of the CAM6 PPE Emulator

7.1. Factors Affecting the Emulator Performance

Some of our target variables are model scores as opposed to raw model output. The calculation (Equation 1) of the model scores compresses data at different latitude bands into one value. This is done so that the systematic uncertainty from satellite observations can be easily taken into account in a global bulk penalty score, with minimization of a bulk score being a more tractable effort for parameter estimation efforts. But, this comes at a cost: a weakened relationship between the parameters and target variables. This scenario appears to be less of an issue in the ModelE3 PPE, but is present in CAM6. Moreover, the model score is a function of the squared

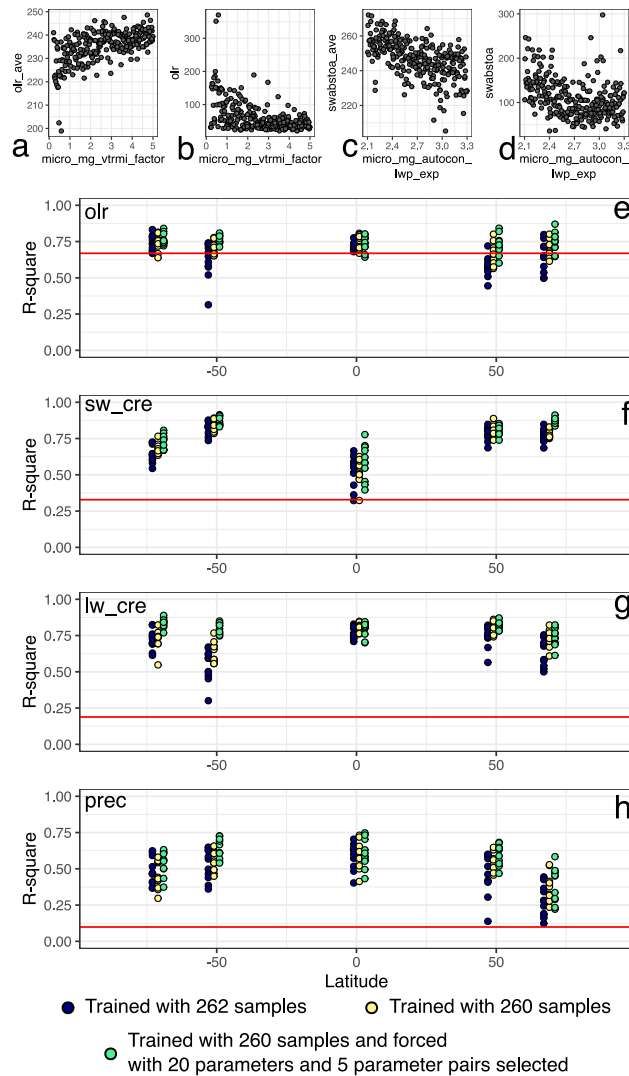


Figure 9. (a–d): Target variables “olr_ave” (model output), “olr” (model score), “swabstoa_ave” (model output), and “swabstoa” (model score) plotted against their most important parameters, respectively, from the CAM6 Perturbed Parameter Ensemble (PPE) Default Test; (e–g): the R-squares of selected variables at different latitudes for the CAM6 PPE. The red horizontal lines denote the R-squares of the corresponding variables in the Default Test. The blue, yellow, and green points correspond to different ways to train the emulator. See text for more details. Their vertical spread denotes the variability from random sampling. The blue, yellow, and green points corresponds to the variable at the same latitude. Their x-axes are shifted slightly to allow for better visualization.

difference between the model output and observation at different latitude bands, which further implies a loss of information about the model output. This can be seen in Figures 9a–9d in which the patterns between the model output and parameters become less evident when output is converted to scores.

There are a few other considerations possibly important for emulator performance. First, if a few outliers exist in the PPE, their inclusion in the training or validation data set will affect the emulator performance. This is especially true for PPEs with fewer samples. For example, Figure 9a shows the presence of two ensemble members with extreme values for the variable *olr* in the CAM6 PPE.

Additionally, as suggested from our analysis of the ModelE3 PPE, a collection of individually less important parameters, when considered together, could have a cumulative impact on emulator performance. The small PPE size might make their effects harder to detect in the case of the CAM6 PPE. If we force *sage* to select additional parameters, there is a chance that the emulator performance can be improved.

7.2. Experiments Yielding Improvement in Emulator Performance

We investigate to what extent addressing the factors discussed above affect the emulator performance. We select four variables and emulate the corresponding model outputs at five different latitudes (-71° , -51° , 1° , 49° , 69°). The variables are those exhibiting different emulator performances in the Default Test (i.e., the R-squares of the selected variables range from 0.09 to 0.66 in the Default Test). Three different ways of training are employed: (a) standard training as described in the method section; (b) exclusion of the two outliers in Figure 9a, and subsequent training and testing of the emulator; and (c) exclusion of the two outliers, and forcing *sage* to select 20 individual parameters and five parameter pairs during training. The latter is employed because, otherwise, it tends to select fewer parameters (less than 15 in all cases) and parameter groups (two to three in all cases), which might omit parameters that contribute information to the emulator. We do not force selection of parameter groups of three, since in the first two cases of training they were rarely selected in the parameter sequences.

Similar to what is done before, 11 tests (with the two outliers taken out in the second and third ways of training) are performed for each way of training the emulator and for each variable at each latitude.

The R-squares of the test results are plotted in Figures 9e–9h by the latitude. The performance of *sage* for the variable *olr* at different latitudes and with different ways of training the emulator is similar to its performance in the Default Test. The overall emulator performance improves noticeably for the other three variables at the selected latitudes compared to the Default Test.

The three different ways of training the emulator lead to slightly different performance. The R-squares from using the first way tends to have greater R-square variability (e.g., *olr* at the latitude of -51° and *prec*, precipitation, at 49°). The second way of training in most cases leads to a slightly improved method performance compared to the first. Forcing more parameters and parameter groups (the third way of training) in the parameter sequences slightly improves the emulator performance for some variables, for example, *olr* and *lw_cre* (top of the atmosphere long wave cloud radiative forcing) at some latitudes.

Figure 9f (*sw_cre*, top of the atmosphere short wave cloud radiative forcing) shows that the emulator performance varies at different latitudes. We also note that the parameters and parameter groups order in the parameter sequences varies by the latitude for the same target variables, suggesting their spatially-varied dependency on different simulated processes and the governing parameters. These suggest the potential of *sage* in drawing more interpretations from climate model PPEs when applied to zonal or local model output.

With the lessons learned above, we calculate the global averages of 12 variables (Table 1) in CAM6 PPE using *sage* and the NN, trained on 210 ensemble members. The two outliers are excluded, and 11 experiments are performed. The resultant R-squares are presented in Figure 4c, where improvement in performance using both emulator methods is noted. The NN in this case has a slightly better performance.

8. Discussion

As an emulator, our new method is shown to perform comparably to a fully connected NN when applied to the two PPEs. The new method also quantifies the importance of parameters and parameter groups during the training and validation. The interpretability and simplicity of the method comes with a very low cost in terms of emulation fidelity compared with NN. Additional insights on emulator design are drawn from comparing the performance of the two methods.

Our work shows that *sage* could have performance comparable to a NN but without the necessity of estimating the GP hyperparameters. From this comparison, we suggest that the relationship between the parameters and target variables learned by both methods is similar, and therefore can be decomposed to low-frequency variability and interaction of only up to a few parameters. We do not need a sophisticated emulator or statistical model to capture such a relationship when the target variables are global climatologies. *sage* does not quantify the uncertainty of the estimate, but the parameter selection process within the method can be coupled with the formal additive GP

method to estimate the hyperparameters of the GP kernels and hence quantify the uncertainty as outlined in Duvenaud et al. (2011). We note that regardless of uncertainty quantification method, the uncertainty in emulator prediction will be difficult to interpret in the context of sparse climate model PPEs. This is because the characteristics of the underlying variability of the PPE data set (e.g., the non-linear relationship in the data) may not be strictly consistent with the assumptions of the emulator method (e.g., Gaussian noise). Moreover, the estimation of the GP hyperparameters is subject to uncertainty (which is also potentially great given the sparsity of PPEs), but its uncertainty is rarely taken into account in the uncertainty quantification. Methods and discussions on the source and measures to quantify the emulator uncertainty specifically for PPE emulators can be found in L. A. Regayre et al. (2018), J. S. Johnson et al. (2018, 2020), Dunbar et al. (2021), Watson-Parris et al. (2021), Watson-Parris (2021), Lopez-Gomez et al. (2022), Elsaesser et al. (2025) and the references therein.

8.1. Performance in the Presence of Parameter Correlation

The PPE parameters are not always independent with each other. A limitation shared by *sage* as well as potentially other emulator methods would occur when they are trained based on parameters that are not uniformly distributed in the parameter space. This could have an impact on the subsequent parameter estimation process using PPE and emulators, which we highlight later in the text. We note here that methodologies such as Unscented Kalman Inversion method, history matching, and calibrated ensemble members have attempted to capture the parameter correlations during the emulator training and parameter tuning based on the PPEs (e.g., J. Johnson et al. (2015), J. S. Johnson et al. (2020), Dunbar et al. (2021), Lopez-Gomez et al. (2022), Elsaesser et al. (2025)).

8.2. Parameter Importance Conditioned on the Parameter Ranges

The performance of an emulator depends on the specific PPE being analyzed. In our experiments, *sage* and NN both exhibit better performance for the ModelE3 PPE compared to the CAM6 PPE. Besides more obvious factors such as the number of ensemble members and differences in the climate models, the different performance is also related to how the parameter ranges are specified. For example, parameters *dcs* and *vf_multi* in the ModelE3 PPE are shown to be important parameters throughout our analysis. Their importance is partially due to their specified ranges, which are relatively greater than other parameters. From a Bayesian perspective, the prior knowledge on *dcs* and *vf_multi* is more poorly-constrained. The concept of parameter sensitivity or importance for a PPE data set is thus conditioned on the ranges of its parameters. The performance of different emulators is only comparable when focusing on the same PPE data set.

9. Implications for Parameter Estimation

Although this work does not focus on parameter estimation, a step after a PPE is generated and the emulator trained, implications and open questions on parameter estimation using PPEs and emulators can be derived based on our analysis.

9.1. Why Individually Less Important Parameters Can Be Cumulatively Important?

From comparing the performance of *sage* and Neural Network, we have proved that individually less important parameters can be cumulatively important (e.g., Figure 7). This is because most of the target variables are global climatologies. They are the weighted average of local climatologies. A parameter could be very important locally, but if the local region has a small area, the importance of this parameter will be reduced when focusing on the global climatologies. We use a simple example to demonstrate this. We focus on the *TLWP* of two areas from the ModelE3 PPE. They are (a) the oceans in the southern hemisphere and north of the Southern Ocean and (b) the Maritime Continent and its nearby ocean (Figure 10a). These two regions are selected as they are different in area and their *TLWP* is likely dominated by different parameters. We emulate the averaged *TLWP* of the two regions and plot the explained variability separately (less important parameters excluded). Two parameters *vterm_env* and *rhcs1* are important for the Maritime Continent but do not contribute to the *TLWP* variability for the other region at all. This shows the potential of more tuning opportunities if we focus on regional climatologies.

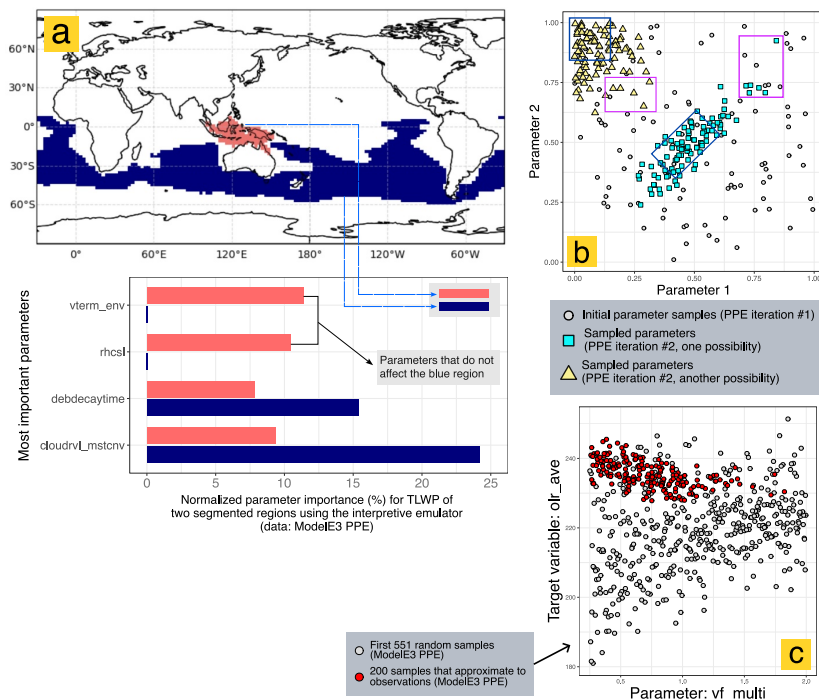


Figure 10. (a) How the parameter importance varies by the region for the variable *TLWP*. Two regions (blue and red masks) are used as targets. The subfigure below the map highlights the parameter importance for the two regional *TLWPs* estimated by *sage*. We use 451 samples for training and 100 samples for testing in this case. Less important parameters and parameter groups not presented for clarity; (b): Possible scenarios of the sampled parameters. The sampled parameters from Perturbed Parameter Ensemble (PPE) iteration #2 shows two possibilities in which the parameters are not evenly distributed in the parameter space; c: target variable *olr_ave* plotted against one important parameter *vf_multi* from the ModelE3 PPE. The first 551 randomly sampled and the last 200 calibrated ensemble members are plotted separately.

9.2. Two Questions Arising From the Iterations of Parameter Estimation Methodologies

The PPE generation, emulator training and data generation, and parameter estimation as a three-step cycle can be iterated for more than once. Each iteration in theory should output sampled parameters that lead to better-matching model outputs when used in the climate model (Figure 10b). It is likely that the samples selected from each iteration are characterized by different parameter-climatogoy relationship (e.g., Parameter A is sensitive in the samples from the first iteration, and it is not sensitive at all in the next) or “non-stationary” behavior (e.g., Parameter A is sensitive locally in the parameter space). In such circumstances, there are two questions to be considered for this methodology.

The sampled parameters can be correlated, unevenly distributed, and clustered around a certain “area” in the parameter space (blue boxes in Figure 10b). When such samples are used to train the emulator for the next iteration, the ones far from the cluster center (magenta boxes in Figure 10b) and have a different local parameter-climatology relationship (if that is the case) with those near the cluster center might degrade the emulator performance, as they are likely weighted less important due to their sparsity during training.

Training the samples from two or more iterations altogether characterized by different parameter-climatogoy relationship resembles mixing heterogeneous and sparse data for training, and hence likely degrades the emulator performance. An example is given in Figure 10c: the relationship between target variable *olr_ave* and *vf_multi* is different for the first 551 (randomly sampled) and last 200 (calibrated to better match observations) ensemble members in the ModelE3 PPE.

We note that the importance of the two questions depends on the goal of the emulation. Their impact will be limited if the goal is to emulate near the Maximum A Posterior, where samples far away from it in the parameter space are not the main focus. See Cleary et al. (2021) for more detailed discussion on this topic.

10. Summary and Conclusions

In this work, we present a new method, referred to as *sage*, that implements analysis and emulation of climate model PPEs simultaneously. This method draws on a simplified version of additive Gaussian Processes, and caters to a key feature of climate model PPEs: a very limited number of ensemble members sampling a high-dimensional parameter space. *sage* identifies and ranks the parameters and parameter groups that affect the value of a target variable, and models their impacts on the target variable as additive terms.

sage is applied to two climate model PPEs (ModelE3 and CAM6), and it performs comparably to a NN. Regarding ModelE3, improved performance is tied to an increase in the number of ensemble members, with further enhancement after accounting for the relationship between target variables during training. For CAM6, *sage* slightly outperforms or has comparable performance (target variable dependent) to a NN. *sage* is designed to be simple and more interpretable, and the comparative study with NN in this study confirms a low cost in terms of emulation fidelity for *sage*. In this work, *sage* is applied to emulate the global averages or averaged zonal skill scores of climate model outputs, although it can be used to emulate zonal or local climatologies.

Upon analyses of the emulator applied to ModelE3 and CAM6, together with comparative study with Neural Network, we learn that the following points are critical for emulator design and emulator-facilitated analyses of PPEs:

1. For sparse climate model PPEs (i.e., limited ensemble members relative to the number of parameters), the relationship between the global target climatologies and parameters captured by an emulator can be mostly decomposed into additive effects of individual parameters and parameter pair interactions. The relationship is also best characterized by low-frequency variation (as opposed to noise-like high frequency variation) of the target variable with respect to the parameters. This argument is supported by the simplicity and additive feature of *sage* and its comparable performance with Neural Network.
2. Neither emulating the variables all at once nor one-at-a-time is (always) the perfect emulator design. Having separate emulators focusing on separate groups of target variables might lead to improved overall performance compared with one emulator simulating all variables at once. The presence of a stand-alone relationship between one or a few target variables and some parameters could be difficult to capture by a NN and potentially other emulation methods if all variables are emulated at once. Incorporating a relationship between target variables into the emulator, if a clear one exists, could further improve emulator performance.
3. A group of individually less important parameters could have a non-negligible and cumulative impact on the overall emulator performance. Thus, we need to be very careful in determining whether a parameter is important or not.
4. Emulator performance may not always increase (linearly) with the number of training ensemble members (e.g., *olr_ave* in Figure 8a), and this is target variable-specific. It is therefore beneficial to generate ensemble members progressively (e.g., 100 at a time), followed by quantification of how performance changes with the presence of more training data, and then a decision on whether to generate more ensemble members.
5. How we define the target variables (i.e., for a given variable, do we formulate as a model score or raw output?) to be emulated may affect emulator performance. There are trade-offs to be made between directly emulating PPE member performance skill versus raw output.

Our work has implications on climate model parameter estimation using the PPE and emulator methodology:

1. The importance of parameters varies by the region of interest. Analyzing such spatial dependency and variability might lead to more effective target variables for parameter estimation.
2. The iterative use of PPE generation, emulator training, and parameter estimation leads to sampled parameters from each iteration. Two questions might arise when the parameter-climatology relationship is non-stationary in the parameter space or different in these iterations: (a) The emulator performance might degrade in the